SMART RECIPE & MEAL PLANNER WEB APPLICATION

(PlanYaChop by GROUP 8)

GROUP MEMBERS

- **Basung Pearly**
- **Bafri Zita**
- **Njong Fabrice**
- **Nkumbe Dilan**

Technology Stack: JAVA (Spring Boot), PostgreSQL, REST API Architecture

TABLE OF CONTENT

1. **INTRODUCTION**
2. **OVERALL SYSTEM DESCRIPTION**
3. **FUNCTIONAL REQUIREMENTS**
4. **NON-FUNCTIONAL REQUIREMENTS**
5. **DATABASE SCHEMA**
6. **API ENDPOINT SUMMARY**
7. **DATA FLOW DIAGRAMS**
8. **STATE TRANSITION DIAGRAM**
9. **FALLBACK STRATEGIES**
10. **ASSUMPTIONS AND DEPENDENCIES**
11. **PROJECT TIMELINE**
12. **APPENDICES**
13. **GLOSSARY**

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements for the Smart Recipe & Meal Planner Web Application, a Java-based system that enables users to generate recipes from available ingredients, create meal plans, and automate shopping list generation using AI.

The purpose of this document is to:

- Serve as a technical blueprint for development
- Define measurable system behaviour
- Ensure alignment between stakeholders and developers

1.2 Scope

The system provides:

- Ingredient-based recipe matching
- Weekly meal planning
- Automated shopping list aggregation
- Nutritional information display
- AI-generated recipe suggestions
- Rating and commenting system
- Role-based access control (User/Admin)

The application will be deployed as a responsive web application using Java Spring Boot for backend services.

2. OVERALL SYSTEM DESCRIPTION

2.1 System Architecture Overview

Architecture Pattern: Layered Architecture (Spring Boot)

Layers:

1. Controller Layer (REST Endpoints)
2. Service Layer (Business Logic)
3. Repository Layer (JPA/Hibernate)
4. Database Layer (PostgreSQL)
5. AI Integration Layer

3. FUNCTIONAL REQUIREMENTS

3.1 User Authentication and Account Management

FR-1: User Registration

The system shall allow users to register by providing email and password.

- Email must be unique.
- Password must contain:
  - Minimum 8 characters
  - One uppercase letter
  - One number
- Password shall be hashed using BCrypt before storage.
- System shall verify email format.
- System shall store user role as USER by default.

FR-2: User Login

The system shall authenticate users using email and password.

- Validate credentials against encrypted password.
- Generate JWT token upon success.
- Token shall include user ID and role.
- Token expiration: 60 minutes.

FR-3: Role-Based Authorization

The system shall restrict certain endpoints to ADMIN users only.

Admin-only actions:

- Delete users
- Delete recipes
- Moderate comments

3.2 Recipe Management

FR-4: Create Recipe

The system shall allow Admin users to create recipes.

Recipe must include:

- Title
- Description
- Ingredient list
- Preparation steps
- Cooking time
- Nutritional values

Validation:

- Title cannot be empty.
- At least one ingredient required.
- Cooking time must be positive.

FR-5: Update Recipe

Authorized users shall modify recipe fields.

System shall:

- Maintain update timestamp.
- Validate ingredient integrity.

FR-6: Delete Recipe

Admin shall delete recipe.
Deletion shall cascade remove related ratings.

FR-7: View Recipe

System shall display:

- Ingredient quantities
- Preparation steps
- Nutrition breakdown
- Ratings and comments

3.3 Ingredient-Based Recipe Matching

FR-8: Ingredient Input

Users shall submit a list of ingredients.

FR-9: Matching Algorithm

System shall:

- Compare input ingredients to recipe ingredients.
- Compute match percentage:
  (Matching Ingredients ÷ Total Required Ingredients) × 100
- Return missing ingredients.

3.4 Weekly Meal Planner

FR-10: Create Weekly Plan

User selects start date.
System creates 7-day structure.

FR-11: Assign Recipe to Day

User selects recipe for specific day.

FR-12: Modify Plan

User edits or removes meal entry.

3.5 Shopping List Generation

FR-13: Auto Generate Shopping List

System shall:

- Extract ingredients from all planned recipes.
- Aggregate identical ingredients.
- Sum quantities.

FR-14: Mark Item as Purchased

User toggles purchased status.

3.6 Ratings and Comments

FR-15: Add Rating

Users rate 1–5 stars.
System recalculates average rating dynamically.

FR-16: Add Comment

Users submit comment.
Admin can moderate.

3.7 AI Recipe Generation

FR-17: AI Custom Recipe

User provides:

- Ingredients
- Dietary preference

System shall:

- Generate recipe
- Estimate nutrition
- Suggest substitutions

4. NON-FUNCTIONAL REQUIREMENTS

4.1 Performance

- API response < 500ms under normal load.
- System shall handle 10,000 concurrent users.
- Database indexed on:
    - user_id
    - recipe_id
    - ingredient_id

4.2 Security

- HTTPS encryption mandatory.
- JWT authentication required.
- CSRF protection.
- SQL Injection prevention via Prepared Statements.
- Password hashing using BCrypt (strength 12).

4.3 Reliability

- 99% uptime.
- Transaction rollback on failure.
- Daily automated backups.

4.4 Scalability

- Stateless REST APIs.
- Horizontal scaling supported.
- Load balancer compatibility.

4.5 Maintainability

- Modular code structure.
- Separation of concerns.
- Clear documentation.

5. DATABASE SCHEMA

Core Tables

USER

Column	Type	Constraint
id	UUID	PK
email	VARCHAR(100)	UNIQUE
password	VARCHAR(255)	NOT NULL
role	VARCHAR(20)	DEFAULT 'USER'
created_at	TIMESTAMP	NOT NULL

RECIPE

id | UUID | PK
title | VARCHAR | NOT NULL

description | TEXT
cooking_time | INT | CHECK > 0
calories | INT

INGREDIENT

id | UUID | PK
name | VARCHAR | UNIQUE

RECIPE_INGREDIENT

Composite PK (recipe_id, ingredient_id)

MEAL_PLAN

id | UUID | PK
user_id | UUID | FK

SHOPPING_LIST

id | UUID | PK

meal_plan_id | UUID | FK

Event Collection (Audit Log)

EVENT_LOG

id | UUID | PK user_id | UUID action | VARCHAR | | timestamp | TIMESTAMP entity_type | VARCHAR entity_id | UUID

Additional Collections

- LOGIN_ATTEMPTS
- PASSWORD_RESET_TOKENS
- AI_REQUEST_LOG
- SYSTEM_ERRORS

6. API ENDPOINT SUMMARY

User Routes

POST /api/auth/register
POST /api/auth/login
GET /api/user/profile
PUT /api/user/profile
DELETE /api/user/account

Admin Routes

GET /api/admin/users
DELETE /api/admin/user/{id}
DELETE /api/admin/recipe/{id}

Recipe Routes

POST /api/recipes
GET /api/recipes
GET /api/recipes/{id}
PUT /api/recipes/{id}
DELETE /api/recipes/{id}

Search Routes

POST /api/search/by-ingredients
GET /api/search/by-name

Meal Plan Routes

POST /api/mealplans
PUT /api/mealplans/{id}
DELETE /api/mealplans/{id}

Shopping Routes

GET /api/shopping/{mealPlanId}
PUT /api/shopping/item/{id}

AI Routes

POST /api/ai/generate
POST /api/ai/optimize-plan

7. DATA FLOW DIAGRAMS

Level 0 DFD (Context Diagram)

Entities:

- User
- Admin
- AI Service

Process:
Smart Recipe System

Data flows:

- Ingredient input
- Recipe suggestions
- Meal plan data
- Shopping list output

Level 1 DFD

Processes:

1. Authentication Module
2. Recipe Management
3. Meal Planner
4. Shopping Aggregator
5. AI Service Handler

Data Stores:

- User DB
- Recipe DB
- Meal Plan DB
- Shopping DB
- Event Log DB

8. STATE TRANSITION DIAGRAM

States:

- Logged Out
- Logged In
- Browsing
- Editing Plan
- Generating AI Recipe
- Viewing Shopping List

Transitions:
Login → Logged In
Logout → Logged Out
Generate Plan → Editing Plan
Generate AI → AI State

9. FALLBACK STRATEGIES

- If AI service unavailable → fall back to rule-based recipe suggestions.
- If database fails → system enters read-only mode.
- If payment API fails (future extension) → retry mechanism.
- If JWT invalid → force re-authentication.

10. ASSUMPTIONS AND DEPENDENCIES

Assumptions:

- Users have stable internet.
- External AI API available.
- PostgreSQL server stable.

Dependencies:

- Spring Boot framework.
- JWT library.
- AI external service.
- Nutrition API.

11. PROJECT TIMELINE

Week 1 – Learning Concepts

Actions:

- Study Spring Boot
- Learn JPA/Hibernate
- Study REST API design
- Study JWT authentication
- Learn PostgreSQL normalization
- Review DFD and UML modelling
- Study AI REST integration

Deliverable:

- Concept summary document
- Draft architecture diagram

Week 2 – Setup & Database Design

Actions:

- Install JDK 17
- Create Spring Boot project
- Setup PostgreSQL
- Design ER diagram
- Normalize database
- Write SQL scripts
- Configure security configuration

Deliverable:

- Database schema
- Running backend skeleton

Week 3 – Backend Development

Actions:

- Implement authentication module
- Implement recipe CRUD
- Implement meal plan logic
- Implement shopping aggregation
- Implement rating system

- Test APIs with Postman

Deliverable:

- Functional backend API

Week 4 – Frontend Development

Actions:

- Design UI wireframes
- Build login/register pages
- Build recipe search page
- Implement planner UI
- Connect to backend APIs

Deliverable:

- Fully integrated frontend

Week 5 – AI Integration

Actions:

- Connect AI service
- Implement AI endpoints
- Implement fall back rules
- Test AI prompt optimization
- Log AI requests

Deliverable:

- AI-enabled system

Week 6 – Testing

Actions:

- Unit tests (JUnit)
- Integration tests
- Security testing
- Performance testing

- Bug fixes

Deliverable:

- Test report

Week 7 – Test Deployment & Documentation

Actions:

- Deploy backend (AWS)
- Deploy frontend
- Configure CI/CD
- Perform UAT
- Finalize documentation
- Prepare presentation

Deliverable:

- Live deployed system
- Final academic submission

12. APPENDICES

Appendix A – Frontend

- HTML5
- CSS3
- JavaScript
- REST API integration

Appendix B – Backend

- Java 17
- Spring Boot
- Spring Security
- JPA/Hibernate

Appendix C – Database

- PostgreSQL
- Normalized schema
- Indexed fields

Appendix D – AI Model

- Prompt-based generation
- Nutrition estimation logic
- Substitution algorithm

Appendix E – External APIs

- Nutrition Data API
- AI Recipe API

Appendix F – DevOps

- GitHub
- CI/CD pipeline
- Docker
- AWS Deployment

13. GLOSSARY

JWT – JSON Web Token
CRUD – Create, Read, Update, Delete
REST – Representational State Transfer
DFD – Data Flow Diagram
ERD – Entity Relationship Diagram